

```

"""
Comparateur de prix - Serveur principal
-----

Un petit serveur web qui cherche un produit et affiche les offres
triées par prix.

- Si les cles eBay sont configurees (variables d'environnement),
  il interroge la vraie API eBay.
- Sinon, il fonctionne en "mode demo" avec des donnees d'exemple,
  pour que tu puisses voir le site marcher tout de suite.
"""

import os
import time
import requests
from flask import Flask, request, jsonify, render_template

app = Flask(__name__)

# --- Cles eBay (a renseigner dans les variables d'environnement Railway) ---
EBAY_CLIENT_ID = os.environ.get("EBAY_CLIENT_ID", "")
EBAY_CLIENT_SECRET = os.environ.get("EBAY_CLIENT_SECRET", "")

# Le "marketplace" eBay a interroger. EBAY_FR = France.
EBAY_MARKETPLACE = os.environ.get("EBAY_MARKETPLACE", "EBAY_FR")

# On garde le token d'accès en memoire avec sa date d'expiration
_token_cache = {"value": None, "expires_at": 0}

def mode_demo_actif():
    """Vrai si on n'a pas de cles eBay : on tourne alors en mode demo."""
    return not (EBAY_CLIENT_ID and EBAY_CLIENT_SECRET)

def obtenir_token_ebay():
    """
    Recupere un token d'accès eBay (valable ~2h) et le met en cache.
    Utilise l'authentification "client credentials" (OAuth).
    """
    # Si on a déjà un token encore valide, on le reutilise
    if _token_cache["value"] and _token_cache["expires_at"] > time.time():
        return _token_cache["value"]

```

```

url = "https://api.ebay.com/identity/v1/oauth2/token"
reponse = requests.post(
    url,
    auth=(EBAY_CLIENT_ID, EBAY_CLIENT_SECRET),
    data={
        "grant_type": "client_credentials",
        "scope": "https://api.ebay.com/oauth/api_scope",
    },
    timeout=15,
)
reponse.raise_for_status()
data = reponse.json()

_token_cache["value"] = data["access_token"]
# On retire 60s de marge de securite avant l'expiration reelle
_token_cache["expires_at"] = time.time() + int(data["expires_in"]) - 60
return _token_cache["value"]

def chercher_sur_ebay(terme, limite=20):
    """
    Interroge la Browse API d'eBay et renvoie une liste d'offres
    simplifiees, trieess par prix croissant.
    """
    token = obtenir_token_ebay()
    url = "https://api.ebay.com/buy/browse/v1/item_summary/search"
    headers = {
        "Authorization": f"Bearer {token}",
        "X-EBAY-C-MARKETPLACE-ID": EBAY_MARKETPLACE,
    }
    params = {
        "q": terme,
        "limit": limite,
        "sort": "price", # tri par prix croissant
    }
    reponse = requests.get(url, headers=headers, params=params, timeout=15)
    reponse.raise_for_status()
    data = reponse.json()

    offres = []
    for item in data.get("itemSummaries", []):
        prix = item.get("price", {})
        offres.append({
            "titre": item.get("title", "Sans titre"),
            "prix": float(prix.get("value", 0)),
            "devise": prix.get("currency", "EUR"),
            "vendeur": item.get("seller", {}).get("username", "Inconnu"),
        })

```

```

        "etat": item.get("condition", "Non precise"),
        "image": item.get("image", {}).get("imageUrl", ""),
        "lien": item.get("itemWebUrl", "#"),
        "source": "eBay",
    })
    return offres

def offres_demo(terme):
    """Donnees d'exemple pour le mode demo (sans cles eBay)."""
    base = [
        {"titre": f"{terme} - tres bon etat", "prix": 24.90, "etat": "Tres bon et"},
        {"titre": f"{terme} - occasion", "prix": 31.00, "etat": "Bon etat"},
        {"titre": f"{terme} - comme neuf", "prix": 39.50, "etat": "Comme neuf"},
        {"titre": f"{terme} - neuf sous blister", "prix": 54.99, "etat": "Neuf"},
        {"titre": f"{terme} - lot de 2", "prix": 45.00, "etat": "Occasion"},
    ]
    offres = []
    for i, o in enumerate(base):
        offres.append({
            "titre": o["titre"],
            "prix": o["prix"],
            "devise": "EUR",
            "vendeur": f"vendeur_demo_{i+1}",
            "etat": o["etat"],
            "image": "",
            "lien": "#",
            "source": "DEMO",
        })
    offres.sort(key=lambda x: x["prix"])
    return offres

@app.route("/")
def accueil():
    return render_template("index.html", mode_demo=mode_demo_actif())

@app.route("/api/recherche")
def api_recherche():
    terme = request.args.get("q", "").strip()
    if not terme:
        return jsonify({"erreur": "Merci d'indiquer un produit a rechercher."}),

    try:
        if mode_demo_actif():
            offres = offres_demo(terme)

```

```

        mode = "demo"
    else:
        offres = chercher_sur_ebay(terme)
        mode = "reel"
    return jsonify({"mode": mode, "terme": terme, "offres": offres})
except requests.HTTPError as e:
    return jsonify({"erreur": f"Erreur cote eBay : {e}"}), 502
except Exception as e:
    return jsonify({"erreur": f"Une erreur est survenue : {e}"}), 500

@app.route("/sante")
def sante():
    """Petite route pour verifier que le serveur tourne."""
    return jsonify({"statut": "ok", "mode_demo": mode_demo_actif()})

if __name__ == "__main__":
    # Railway fournit le port via la variable d'environnement PORT
    port = int(os.environ.get("PORT", 8080))
    app.run(host="0.0.0.0", port=port)

```